Experiment Title: File and Directory Management in Linux Terminal

Topic: Creating, Copying, Moving, and Removing Files and Directories

1. Objective

The objective of this lab is to learn how to create, duplicate, relocate, and delete files and directories using the Linux command-line interface. By completing this experiment, we aim to:

- Create single and nested folders using **`mkdir`** and **`mkdir -p`**.
- Create new empty files quickly using the **`touch`** command.
- Duplicate files into new locations or new names using **`cp`**.
- Move files between folders and rename existing files using **`mv`**.
- Safely delete files and directories using **`rm`**, **`rm -r`**, and **`rm -rf`**.

2. Equipment / Software Requirements

Item	Details	Item	Details
Operating System	Linux (NixOS 26.11, Kernel 6.12)	**Host Machine**	`firefly`
User Account	`ratul`	**Command Shell**	GNU Bash 5.2 / Zsh
Terminal Emulator	Linux Terminal (Kitty / Ghostty)	**Core Tools**	GNU Coreutils (`mkdir`, `touch`, `cp`, `mv`, `rm`)

3. Theoretical Background

File and directory management is the most essential skill in Linux. Unlike graphical file managers where users click and drag icons, the terminal allows us to organize, copy, and clean up files very quickly using short commands.

3.1 Commands Practiced in This Lab

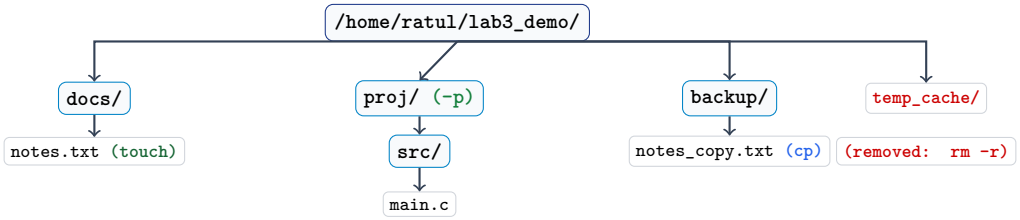
Command	Syntax	Simple Meaning / Purpose	Example
`mkdir`	`mkdir <name>`	Creates a new empty directory (folder).	`mkdir docs`
`mkdir -p`	`mkdir -p <path>`	Creates parent and child folders together in one step.	`mkdir -p proj/src/utils`
`touch`	`touch <file>`	Creates a new empty file, or updates timestamp.	`touch notes.txt`
`cp`	`cp <src> <dest>`	Copies a file to a new name or destination folder.	`cp notes.txt backup.txt`
`mv`	`mv <src> <dest>`	Moves a file to another folder, or renames it.	`mv old.txt new.txt`
`rm`	`rm <file>`	Deletes one or more files permanently.	`rm temp.log`
`rm -r`	`rm -r <folder>`	Deletes a folder and all its contents recursively.	`rm -r old_data`
`rm -rf`	`rm -rf <path>`	Forcefully deletes folder and files without any prompt.	`rm -rf build`

3.2 Important Rules for File Operations

- **No Recycle Bin in Terminal:** Commands like **`rm`** delete files immediately and permanently. There is no graphical Trash folder to recover deleted files.
- **Parent Flag (`-p`):** Standard **`mkdir a/b/c`** fails if folder **`a`** does not exist yet. Adding **`-p`** tells Linux to build all missing parent folders automatically.
- **Recursive Flag (`-r`):** Standard **`rm`** refuses to remove directories. The **`-r`** (recursive) flag dives inside the directory and removes all files and subfolders first.
- **Force Flag (`-f`):** Suppresses all warning messages and ignores non-existent files. Combined as **`rm -rf`**, it forces complete removal without stopping.

4. Equipment / Demo Directory Setup

To practice these commands safely, we created an isolated practice folder called **lab3_demo** inside **/home/ratul**. The diagram below shows the structure we constructed, organized, and cleaned during this lab:



5. Procedure / Steps

We carried out the file and directory management tasks using the following simple steps:

1. Created our practice workspace folder **lab3_demo** using **mkdir** and entered it with **cd**.
2. Created a simple directory called **docs** using **mkdir**.
3. Created a multi-level nested folder tree **proj/src/utils** in one single command using **mkdir -p**.
4. Created new empty text files (**notes.txt** and **main.c**) using the **touch** command.
5. Made a backup folder and duplicated **notes.txt** into **notes_copy.txt** using **cp**.
6. Moved **notes_copy.txt** into the **backup/** folder and renamed files using **mv**.
7. Deleted a single unwanted file safely using **rm**.
8. Created a temporary folder containing files and deleted it completely using **rm -r**.
9. Tested forced recursive deletion of stubborn folder trees using **rm -rf**.

6. Result / Output Demonstration

All commands were executed under user account **ratul** on host machine **firefly**.

6.1 Session 1: Creating Directories, Nested Trees, Empty Files, and Copying

ratul@firefly: ~/lab3_demo — bash

```
ratul@firefly:~/lab3_demo$ mkdir docs backup
ratul@firefly:~/lab3_demo$ mkdir -p proj/src/utils
ratul@firefly:~/lab3_demo$ touch docs/notes.txt proj/src/main.c
ratul@firefly:~/lab3_demo$ ls docs proj/src
docs:  notes.txt        proj/src:  main.c  utils
ratul@firefly:~/lab3_demo$ cp docs/notes.txt docs/notes_copy.txt
ratul@firefly:~/lab3_demo$ ls -l docs
total 0
-rw-r--r- 1 ratul users 0 Oct  8 14:10 notes.txt
-rw-r--r- 1 ratul users 0 Oct  8 14:12 notes_copy.txt
ratul@firefly:~/lab3_demo$ _
```

Explanation of Session 1 Results:

- **mkdir** and **mkdir -p**: Created **docs** and **backup**. With **-p**, Linux automatically created the whole chain **proj/src/utils** without complaining that parent folders were missing.
- **touch**: Instantly created new files **notes.txt** and **main.c** with 0 bytes size.
- **cp**: Duplicated **notes.txt** into **notes_copy.txt**. Both files now exist independently in the folder.

6.2 Session 2: Moving, Renaming, and Removing Files and Directories

ratul@firefly: ~/lab3_demo — bash

```
ratul@firefly:~/lab3_demo$ mv docs/notes_copy.txt backup/
ratul@firefly:~/lab3_demo$ mv backup/notes_copy.txt backup/notes_v1.txt
ratul@firefly:~/lab3_demo$ ls backup
notes_v1.txt
ratul@firefly:~/lab3_demo$ touch temp.log && rm temp.log
ratul@firefly:~/lab3_demo$ mkdir -p temp_cache/data && touch temp_cache/data/cache.bin
ratul@firefly:~/lab3_demo$ rm -r temp_cache
ratul@firefly:~/lab3_demo$ mkdir -p scratch/a/b && touch scratch/a/test.tmp
ratul@firefly:~/lab3_demo$ rm -rf scratch
ratul@firefly:~/lab3_demo$ ls
backup   docs   proj
ratul@firefly:~/lab3_demo$ _
```

Explanation of Session 2 Results:

- `mv` for moving and renaming: When given a destination folder (`backup/`), `mv` relocated the file. When given a new file name (`notes_v1.txt`), `mv` renamed the file in place.
- `rm`: Deleted the single file `temp.log` instantly.
- `rm -r`: Successfully deleted the non-empty folder `temp_cache` by removing its subfolder and file recursively.
- `rm -rf`: Forcibly deleted the entire `scratch` hierarchy without asking confirmation or throwing errors.
- Final verification with `ls` confirmed our workspace remained clean with only `backup`, `docs`, and `proj`.

7. Conclusion

In this lab, we practiced the primary commands used for everyday file and directory management in Linux. We learned:

1. How to create single folders with **`mkdir`** and nested folder paths with **`mkdir -p`**.
2. How to generate files with **`touch`** and duplicate them safely using **`cp`**.
3. How to relocate and rename files efficiently using the **`mv`** command.
4. How to delete files with **`rm`**, and recursively remove directory structures using **`rm -r`** and **`rm -rf`**.

Understanding these commands enables fast and structured organization of files in the Linux operating system.